Test Plan

1. Project Scope

1.1 Background

`ARG-Test` is an AI-driven AutoTestDesign tool that converts natural-language requirements into auditable black-box test suites. The final project extends the middle-phase prototype into a submission-quality system by adding:

- requirement structuring
- risk analysis and prioritization
- multi-technique black-box generation
- state-model extraction and sequence planning
- detailed executable evidence for one major module
- non-functional validation
- reproducibility and replay support

1.2 Objectives

The objectives of the final project are:

- generate requirement-driven test cases in a structured, reviewable form
- align the generated suites with classical testing techniques rather than free-form prompting only
- provide risk-aware prioritization so that test effort can be focused
- export reusable testing artifacts in standard formats
- demonstrate one selected module with both black-box and white-box execution evidence
- produce a submission package that is auditable and presentation-ready

1.3 In-Scope and Out-of-Scope Items

In scope:

- requirement ingestion from file, direct text, and CSV batch input
- trace generation, parsing, checking, reranking, repair, risk scoring, and export
- comparison against rule-based, plain-LLM, and structured-no-checker baselines
- detailed design and execution for `coupon_discount_engine`

Out of scope:

- industrial-scale load testing
- UI-centered usability studies with external participants

- full determinism claims at the upstream provider level

2. Test Items

2.1 Major Functional Features

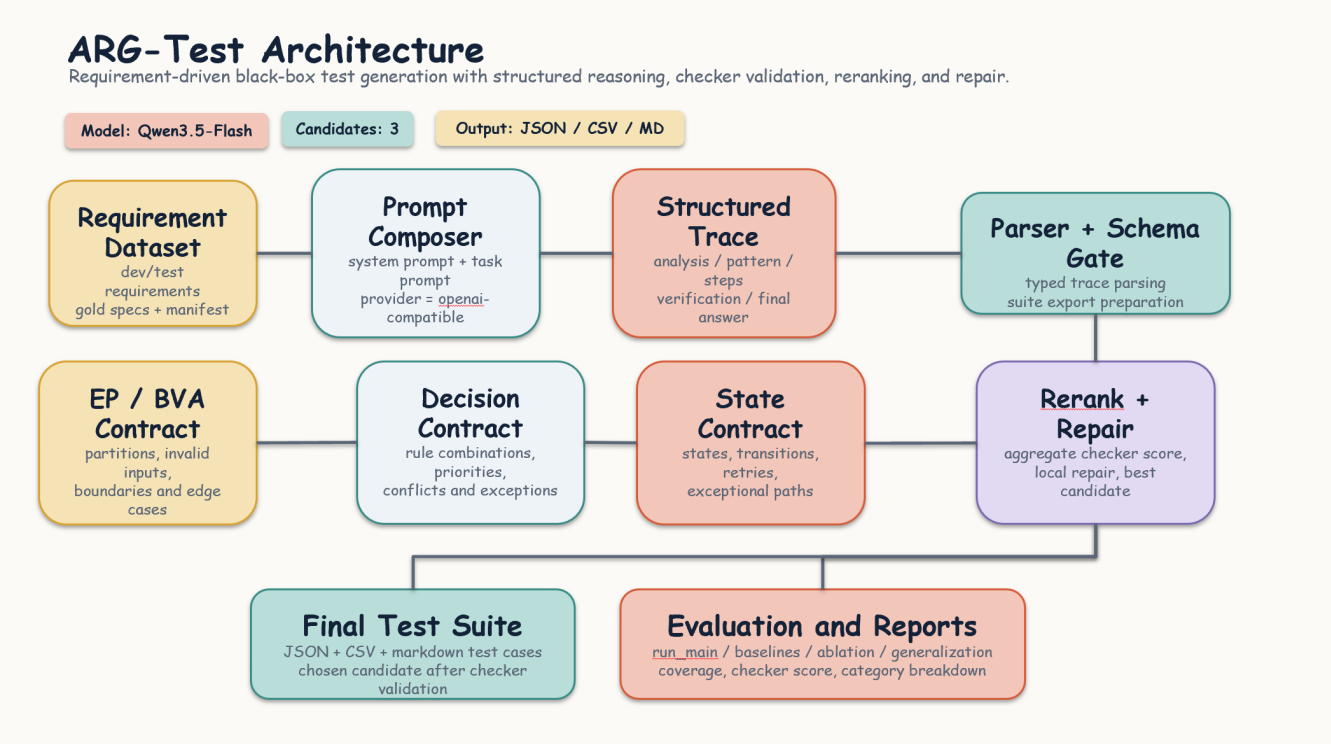
Feature	Description	Primary repository modules
Requirement input/parsing	Read raw requirements from file, direct text, or CSV and structure them for downstream testing logic.	`src/main.py`, `src/input_loader.py`, `src/parser.py`
Structured trace generation	Generate five-part reasoning traces for requirement-driven test design.	`src/pipeline.py`, `src/llm_client.py`
Technique-aware checking	Verify EP, BVA, decision-table, and state-transition obligations.	`src/checker/`
Candidate selection and repair	Select the strongest candidate and repair missing obligations when needed.	`src/reranker.py`, `src/repair.py`
Risk scoring and prioritization	Assign requirement-level risk scores and promote testing focus accordingly.	`src/risk.py`
State-model extraction	Extract states, legal/illegal transitions, and sequence plans.	`src/state_model.py`
Export and reporting	Export Markdown, JSON, CSV, manifests, and aggregate summaries.	`src/exporter.py`, `src/evaluation/metrics.py`

2.2 Major Non-Functional Features

Non-functional feature	Why it matters in this project	Main evidence
Performance	The tool should finish small to medium requirement batches fast enough for practical coursework use.	Mock runtime checks
Usability	The tool should support direct text, file, and CSV workflows and produce readable outputs.	CLI commands, `README`, exported artifacts
Security	Result bundles and report assets must not leak secrets such as API keys.	Secret-leak scan and manifest review
Maintainability	The repository should remain modular and testable after the final upgrade.	`src/`, `experiments/`, `tests/`, regression suite

Non-functional feature	Why it matters in this project	Main evidence
Reproducibility	Final results should be reconstructable even when the live provider is not perfectly deterministic.	repeatability summaries and replay support

2.3 System Architecture and Main Components



ARG-Test Architecture

The architecture is centered on a requirement-driven pipeline:

1. ingest a requirement from file, direct text, or CSV
2. generate multiple structured traces
3. parse and validate each candidate
4. rerank or repair when necessary
5. enrich the selected output with risk and state-model metadata
6. export final suites and aggregate reports

Main components:

- `src/main.py`: CLI entry point
- `src/pipeline.py`: orchestration of generation, checking, selection, and enrichment
- `src/llm_client.py`: provider-facing model wrapper
- `src/parser.py`: typed structured-trace parser
- `src/checker/`: testing-theory contract checkers

- `src/reranker.py` and `src/repair.py`: candidate control
- `src/risk.py`: risk scoring logic
- `src/state_model.py`: state extraction and coverage-plan generation
- `src/exporter.py`: artifact export
- `src/evaluation/metrics.py`: coverage and checker-based evaluation

3. High-Level Test Strategy

The overall strategy is risk-aware and evidence-driven. High-risk business-rule and workflow requirements receive stronger attention in both the main experiments and the detailed execution module.

3.1 Test Suites

Suite ID	Suite name	Objective	Main techniques	Evidence
A	Functional pipeline validation	Verify end-to-end generation, parsing, checking, reranking, repair, and export.	integration testing, script-based system checks	`experiments/run_main.py`, report bundles
B	Output quality evaluation	Measure checker score, overall coverage, duplicates, and diagnostics on frozen test requirements.	gold-spec-based evaluation	`outputs/reports/`, formal summary tables
C	Baseline comparison	Compare the full pipeline with non-AI and weaker AI alternatives.	controlled comparative experiment	baseline summary and main comparison
D	Ablation	Isolate the effect of checker-guided control over structure-only generation.	component ablation	main report Section 6.4
E	Detailed module execution	Demonstrate black-box plus white-box test design for a selected major module.	EP, BVA, decision-table, white-box branch testing	`coupon_discount_engine` evidence
F	NFR and reproducibility validation	Check maintainability, usability, stability, and replayability of the final package.	regression tests, manifest checks, repeatability runs	`final_docs/execution_evidence/`

3.2 Technique Selection by Risk

Requirement class / risk profile	Selected techniques	Rationale
Input validation with thresholds	EP + BVA	Strong partition and boundary obligations
Business-rule logic with interacting conditions	EP + decision table + targeted BVA	Good fit for rule combinations and monetary thresholds
Workflow/state behavior	state-transition testing + negative transition checks	Necessary to cover legal and illegal transitions
Selected detailed module	EP + BVA + decision table + white-box branch testing	Meets the final-project requirement for mixed black-box and white-box evidence

4. Test Levels and Objectives

Test level	Objective	Typical assets	Exit condition
Unit	Validate helper functions, parser behavior, and checker logic.	targeted Python tests	no critical unit failure
Integration	Verify that generated traces flow correctly through parsing, checking, repair, and export.	pipeline scripts and intermediate artifacts	full pipeline completes without structural failure
System	Validate complete requirement-to-suite behavior on the frozen evaluation set.	formal report bundles	canonical summaries generated successfully
Acceptance	Verify that the final package supports report, PPT, demo, and Q&A needs.	`final_docs`, figures, report assets, evidence summaries	all cited artifacts are present and traceable

5. Schedule and Checklist

The project schedule is organized by deliverable-oriented phases rather than by isolated coding tasks.

Phase	Main work	Deliverables	Status expectation
Phase 1	freeze middle baseline, move final materials into the main repository, confirm canonical result roots	`final_docs/`, `frozen_middle/`, evidence source map	completed before final writing

Phase	Main work	Deliverables	Status expectation
Phase 2	close feature gaps such as risk scoring, CSV/direct input, state-model closure, and reproducibility controls	upgraded pipeline and manifests	completed before formal report writing
Phase 3	implement detailed module evidence, run coverage and mutation checks, and refresh figures	executable evidence and report-ready figures	completed before document finalization
Phase 4	finalize independent documents, align report/PPT wording, and prepare submission package	report PDF, test-plan PDF, risk-report PDF, detailed-execution PDF	pre-submission phase
Phase 5	final consistency review and presentation preparation	PPT PDF, demo video, compressed scripts	final delivery phase

Checklist before submission:

- all cited numbers come from canonical result roots
- final report, risk report, test plan, and detailed execution document use the same terminology
- detailed module evidence paths are reachable from the repository
- PPT and demo use the same selected examples as the report

6. Organization Chart and Responsibilities

Member	Student ID	Primary responsibility
Yi Xu	2351441	group leader; final integration, pipeline control, evidence consistency, final merge
Luowu Zhang	2352746	final report editing, PPT production, presentation assets
Xiang Wang	2351039	dataset maintenance and main experiment support
Fengxuan Kang	2350283	baseline support and detailed module execution support
Yiwei Chen	2350217	evaluation support, ablation/generalization support, validation materials

This organization is important for the final package because the submission is not only code. It also includes formal documents, evidence paths, and presentation assets that must remain consistent with each other.

7. Testing Framework and Rationale

7.1 Primary Framework

`pytest` is the primary execution framework for the detailed module and repository regression checks.

Rationale:

- it matches the Python-based repository directly
- it is lightweight and easy to reproduce on another machine
- it integrates naturally with `coverage`
- it supports selective execution for black-box and white-box cases

7.2 Supporting Toolchain

Tool	Purpose
`coverage.py`	statement and branch coverage collection
repository scripts in `experiments/`	formal experiment orchestration, baselines, repeatability, and figure generation
JSON/CSV/Markdown exports	structured outputs for inspection and downstream reuse

8. Cost Estimation

The cost estimate is intentionally presented as person-days plus tool/runtime overhead rather than vague prose.

| Work item | Estimated effort |

| --- | ---: |

| repository restructuring and baseline freezing | 1.0 person-day |

| final feature upgrades and verification | 2.0 to 3.0 person-days |

| risk analysis and test plan authoring | 1.0 to 1.5 person-days |

| detailed module implementation and execution | 2.0 person-days |

| figure generation, report polishing, and package consistency review | 1.5 to 2.0 person-days |

| PPT, demo, and final packaging | 1.0 to 1.5 person-days |

Estimated total:

- `8.5 to 11.0 person-days`
- moderate API/runtime cost, bounded by frozen results and replay-oriented reuse

9. Exit Criteria

The test plan is considered satisfied when:

- the canonical formal result bundle is fixed and fully traceable
- the final report, risk report, test plan, and detailed execution document are all complete
- the selected major module has both black-box and white-box execution evidence
- NFR and reproducibility evidence are present
- the final package is presentation-ready and can support Q&A without path confusion

10. Conclusion

This test plan is stronger than a minimal coursework plan because it is tied to concrete repository assets, formal evidence roots, named responsibilities, and explicit exit criteria. It therefore functions not only as a planning document, but also as a control document for keeping the final submission coherent.